# CIRCULAR QUEUE

## What is a Circular Queue?

A Circular Queue is a linear data structure that follows the FIFO (First In, First Out) principle.

It is an improved version of a normal queue where the last position is connected back to the first position to form a circle.

In a normal queue, once the rear reaches the last index, no more insertions are possible even if there are empty spaces at the beginning.

Circular Queue solves this problem by reusing empty spaces.

# Why Circular Queue is Better than Normal Queue?

## Problem in Normal Queue

Consider an array queue of size 5.

### Initial Queue

Front → [10] [20] [30] [40] [50] ← Rear

Now perform two dequeue operations:

Front → [ ] [ ] [30] [40] [50] ← Rear

Even though two spaces are empty, insertion is not possible because rear already reached the last index.

This causes:

- Memory wastage
- False overflow condition

# Solution in Circular Queue

In Circular Queue:

- Rear moves circularly to the beginning
- Empty spaces are reused

After dequeue:

[ ] [ ] [30] [40] [50]

New insertions happen at beginning:

[60] [70] [30] [40] [50]

Thus, memory is efficiently utilized.

# Advantages of Circular Queue Over Normal Queue

## 1. Better Memory Utilization

Unused spaces are reused efficiently

## 2. Avoids False Overflow

Queue does not become full unnecessarily

## 3. Faster Operations

Insertion and deletion remain O(1)

## 4. Suitable for Continuous Processing

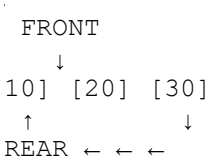
Useful in buffering and scheduling systems

# Basic Operations of Circular Queue

1. **Enqueue** – Insert element at rear
2. **Dequeue** – Remove element from front
3. **Peek** – View front element
4. **isEmpty** – Check whether queue is empty
5. **isFull** – Check whether queue is full

# Representation of Circular Queue

**Example:**




The rear wraps around to the beginning.

# Conditions in Circular Queue

## Queue is Empty When

```
front == -1
```

## Queue is Full When

```
(front == (rear + 1) % MAX)
```

# Circular Queue Using Array in C

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;
```

# Enqueue Operation

```
void enqueue(int x) {

    if ((rear + 1) % MAX == front) {
        printf("Queue Overflow\n");
    }

    else {

        if (front == -1)
            front = 0;
```

```
        rear = (rear + 1) % MAX;
        queue[rear] = x;
    }
}
```

# Dequeue Operation

```
void dequeue() {

    if (front == -1) {
        printf("Queue Underflow\n");
    }

    else {

        printf("Deleted element: %d\n", queue[front]);

        if (front == rear) {
            front = rear = -1;
        }

        else {
            front = (front + 1) % MAX;
        }
    }
}
```

# Peek Operation

```
void peek() {

    if (front == -1)
        printf("Queue is empty\n");

    else
        printf("Front element: %d\n", queue[front]);
}
```

# Display Operation

```
void display() {

    if (front == -1) {
        printf("Queue is empty\n");
```

```
    }

    else {

        int i = front;

        printf("Queue elements:\n");

        while (i != rear) {
            printf("%d\n", queue[i]);
            i = (i + 1) % MAX;
        }

        printf("%d\n", queue[rear]);
    }
}
```

# Characteristics of Circular Queue

- Follows FIFO principle
- Rear connects back to front
- Efficient memory utilization
- Insertions and deletions take O(1) time
- Implemented mainly using arrays

# Real-Life Applications of Circular Queue

## 1. CPU Scheduling

Used in Round Robin scheduling

## 2. Keyboard Buffers

Stores keyboard input continuously

## 3. Printer Queue

Efficient handling of print jobs

## 4. Traffic Management Systems

Vehicles processed cyclically

## 5. Streaming Applications

Continuous buffering of data

# Key Points to Remember

- Circular Queue is an improved version of normal queue
- Rear wraps around using modulo operation %
- Prevents memory wastage
- Efficient for continuous insertion and deletion